Mini Reproduction Experiment: Diagnosing PPO on Andrews–Curtis

Scaled-Down Ablation Study — 147 Instances, 10M Steps, Run 2 Breakthrough

AC-Solver Project

April 2026

Outline

Motivation: Why a Mini Experiment?

The Problem: PPO vs. Paper Results

Our full-scale reproduction (100M steps, 1190 instances) showed:

Algorithm	Solved	Rate	Paper
Greedy (10^6 nodes)	**554** / 1190	46.6%	533 (44.8%)
BFS (10^6 nodes)	278 / 1190	23.4%	278 (23.4%)
PPO (best-of-5, 50M)	32 / 1190	2.7%	431 (36.2%)

Classical search: reproduced (BFS exact, greedy +21).
PPO: 13× gap. Why?

Goal: Identify the root cause(s) efficiently, without another 60-hour run.

Mini Experiment Design Philosophy

Full experiment:

- 1190 instances
- 100M steps (∼60 h)
- 1 configuration tested
- 1 data point

Mini experiment:

- **147 instances** (12.4%)
- **10M steps** (∼6 h each)
- **3 configurations** tested
- 3 data points, controlled comparisons

Key Insight
3 runs × 10M steps = 30M total steps = cost of less than half a single full run.
We get 3 controlled comparisons instead of 1 expensive data point.

Spec: `spec/2026-04-03-scaled-experiment.md`

Systematic Diagnosis

Root Causes, Ranked by Empirical Impact

#	Cause	Impact	Evidence
1	`update_epochs` $= 1$	Single data pass	**Run 2 breakthrough**
2	Reward $\div 14{,}400$	$+1000 \rightarrow +0.069$	Not in paper §4.3
3	Batch $3.5\times$ smaller	Noisier gradients	8 envs vs. 28
4	No variable horizon	Can't solve long-path	Paper: $200 \rightarrow 1200$

Primary finding: `update_epochs=4` was the critical fix. Run 2 (reward fix + epochs $= 4$) solved **8/147 deterministic** and **10/147 stochastic** at 10M steps — while Runs 0 and 1 (epochs $= 1$) solved 0 deterministic each.

Secondary: The reward fix improved the critic (expl. var $-1.7 \rightarrow 0.0$) but did *not* improve the policy by itself (Run 1 $\approx$ Run 0 in evaluation).

Terminology: Deterministic vs. Stochastic Evaluation

After training, we evaluate the learned policy in two modes:

Deterministic (argmax):

- At each step, choose the action with the **highest probability**: $a_t = \arg\max_a \pi_\theta(a \mid s_t)$
- Tests what the policy *learned* — its "best guess" at each state
- **Reproducible**: same state always produces the same action
- Strictest test of policy quality

Stochastic (best-of-k):

- Sample actions from the probability distribution: $a_t \sim \pi_\theta(\cdot \mid s_t)$
- Run k independent rollouts per instance, report the best result
- Allows the policy to **explore** alternatives via randomness
- More forgiving: can "get lucky" on a good trajectory

The Reward Attenuation Problem

The paper defines (Section 4.3):

$$R(s_t, a_t, s_{t+1}) = \begin{cases} -\min(10,\ \text{length}(s_{t+1})) & \text{if } \text{length}(s_{t+1}) > 2 \\ +1000 & \text{otherwise (solved)} \end{cases}$$

No further normalisation. PPO sees rewards in $[-10, +1000]$.

Our code adds one extra line not in the paper:

rewards $/=$ `max_reward` $= 14{,}400 \quad \Longrightarrow \quad$ PPO sees $[-0.0007,\ +0.069]$

	Paper	**Our code**
Success reward	$+1000$	$+0.069$
Per-step penalty	-10	-0.0007
Ratio	$100\times$	$100\times$
Magnitude	Large	$14{,}400\times$ smaller

Worked Example: Why Normalisation Was Added (and Why It's Wrong)

The `max_reward` constant comes from the environment's maximum possible single-step reward:

$$\texttt{max\_reward} = T \times \texttt{max\_relator\_length} \times 2 = 200 \times 36 \times 2 = 14{,}400$$

This is the theoretical maximum return if *every step gave the highest reward*. It was intended to normalise returns to $[0, 1]$ — a common RL practice.

The problem: The actual reward range is $[-10, +1000]$ (after clipping), not $[0, +14{,}400]$. The normalisation constant was derived from the *un-clipped* reward scale, but the code also applies clipping:

Stage	Success	Per-step	Code
1. Raw reward	$+14{,}400$	-10	`ACEnv.step()`
2. Clip $[-10, 1000]$	$+1{,}000$	-10	`clip_rewards=True`
3. $\div 14{,}400$	$+0.069$	-0.0007	`rewards /= max_reward`

Step 2 discards 93% of the signal ($14{,}400 \to 1{,}000$).

Worked Example: A Concrete Trajectory

Consider an episode where the agent solves a presentation at step $t = 80$:

Rewards along the trajectory (presentation length decreasing from 12 to 2):

$$r_0 = -10,\ r_1 = -10,\ \ldots,\ r_{79} = -8,\ r_{80} = +1000$$

Cumulative return (from step 0, with $\gamma = 0.999$):

$$G_0 = \sum_{t=0}^{80} \gamma^t r_t \approx \underbrace{-10 \cdot \frac{1 - 0.999^{80}}{1 - 0.999}}_{\text{penalties} \approx -770} + \underbrace{0.999^{80} \cdot 1000}_{\text{success} \approx +923} \approx +153$$

	Paper scale	**After** $\div 14{,}400$
G_0 (step 0)	$+153$	$+0.0106$
G_{79} (step 79)	$+992$	$+0.0689$
Unsolved trajectory G_0	$-1{,}993$	-0.138

Quantifying the Signal Loss

With $\gamma = 0.999$ and a success at step t:

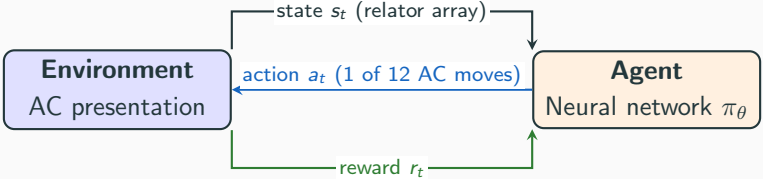
	Paper	**Our code**	**Interpretation**
Raw return at t=0	$+1000$	$+0.069$	
Discounted at t=50	$+951$	$+0.066$	Still strong vs. still tiny
Discounted at t=100	$+905$	$+0.063$	Clear signal vs. noise
Per-step penalty	-10	-0.0007	
SNR at t=100	~ 90	~ 90	Ratio same, scale not
Critic init scale	$O(1)$		Ortho init, gain = 1.0
Critic target	$O(1000)$	$O(0.07)$	Critic already "accurate"

Key insight: PPO trains a **critic** network to predict future returns from each state (details in the PPO section). The critic's random initial predictions are $O(1)$, which is already $14\times$ the actual returns. The critic has no gradient signal to improve because its

PPO Algorithm Review

Step 1: The RL Loop as a Markov Decision Process

We cast AC-trivialization as an MDP (S, A, R, P, ρ):



- **State** s_t: a balanced presentation $\langle x, y \mid r_0, r_1 \rangle$ encoded as an integer array
- **Action** $a_t \in \{0, \ldots, 11\}$: one of 12 AC moves (see next slide)
- **Reward** $r_t = R(s_t, a_t, s_{t+1})$: depends on the resulting presentation length
- **Goal**: find a policy π_θ that maximizes cumulative return:

$$R(\tau) = \sum_{t=0}^{T} \gamma^t \, R(s_t, a_t, s_{t+1}), \qquad \gamma = 0.999$$

The 12 AC Moves (Action Space)

The original AC moves (Section 2 of the paper) are:

(AC1) $r_i \to r_i r_j^{\pm 1}$ for $i \neq j$ (concatenation)

(AC2) $r_i \to r_i^{-1}$ (inversion — dropped: doesn't change length $\Rightarrow$ no signal)

(AC3) $r_i \to g\, r_i\, g^{-1}$ where g is a generator or its inverse (conjugation)

The number of moves depends on the number of **generators** n_g and **relators** n_r:

$$|\text{AC moves}| = \underbrace{n_r(n_r-1)\cdot 2}_{\text{AC'1: concatenations}} + \underbrace{n_r \cdot 2n_g}_{\text{AC'2: conjugations}} = 2n_r(n_r-1) + 2n_r n_g$$

For our setting ($n_g = 2$ generators $\{x, y\}$, $n_r = 2$ relators $\{r_0, r_1\}$): $= 2 \cdot 2 \cdot 1 + 2 \cdot 2 \cdot 2 = 4 + 8 = 12$ moves.

Note: The paper (p7) states "$3n^2$ AC moves" for n generators. With $n = 2$: $3 \cdot 4 = 12$, consistent. For a 3-generator problem, this would be $3 \cdot 9 = 27$ moves — the action space grows quadratically.

The 12 AC Moves: Concatenations and Conjugations

Concatenations (4 moves)

ID	Move
0	$r_1 \to r_1 \cdot r_0$
1	$r_0 \to r_0 \cdot r_1^{-1}$
2	$r_1 \to r_1 \cdot r_0^{-1}$
3	$r_0 \to r_0 \cdot r_1$

Conjugations (8 moves)

ID	Move
4	$r_1 \to x^{-1}\, r_1\, x$
5	$r_0 \to y^{-1}\, r_0\, y$
6	$r_1 \to y^{-1}\, r_1\, y$
7	$r_0 \to x\, r_0\, x^{-1}$
8	$r_1 \to x\, r_1\, x^{-1}$
9	$r_0 \to y\, r_0\, y^{-1}$
10	$r_1 \to y\, r_1\, y^{-1}$
11	$r_0 \to x^{-1}\, r_0\, x$

- After each move, **free reduction** cancels adjacent inverse pairs (e.g., $xx^{-1} \to \varepsilon$).
- **Cyclic reduction** cancels at word boundaries (equivalent to conjugation).
- If a move would exceed `max_relator_length` $= 36$, it is **rejected** (a "no-op" — the state is returned unchanged).
- In practice, $\sim$60–70% of actions are no-ops (many moves violate the length bound).

Step 2a: The Policy Gradient Theorem

Policy $\pi_\theta(a \mid s)$: a neural network that outputs a probability distribution over the 12 AC moves, given a presentation state s.

Goal: find θ that maximises the expected return $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$.

The Policy Gradient Theorem (Williams 1992, Sutton et al. 1999) shows:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t, a_t)\right]$$

Two key terms:

- $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$: the **score function** — the direction in parameter space that makes action a_t more likely from state s_t.
- $A^\pi(s_t, a_t)$: the **advantage** — "how much better was action a_t compared to the *average* action from state s_t?"

Step 2b: Why Vanilla Policy Gradient Fails

Problem: vanilla policy gradient can make **catastrophically large updates**.

Concrete example from our AC problem:

Suppose the current policy gives move 3 (concatenation $r_0 \to r_0 r_1$) probability $\pi(a_3 \mid s) = 0.10$ (roughly uniform across 12 moves). The advantage estimate says this move was very good: $A = +5$.

A single gradient step on $L^{PG} = \log(0.10) \times 5 = -11.5$ pushes θ to increase $\pi(a_3 \mid s)$. With a large learning rate, one step might give:

$$\pi(a_3 \mid s): \quad 0.10 \;\to\; 0.85$$

What if the advantage estimate was wrong? (Noisy critic, small batch, early training — all common.) The policy now confidently chooses move 3 from state s and may **never explore alternatives**, leading to permanent failure.

Step 2c: Three Approaches to Constraining Updates

The core problem: the gradient $\nabla_\theta L^{PG}$ tells us a *direction* to improve, but gives no guidance on *how far* to step.

Three approaches to constraining the step size:

1. **Small learning rate:** simple but too conservative — wastes data by making tiny updates even when larger ones are safe.
2. **TRPO** (Schulman et al., 2015): constrain the KL divergence $D_{KL}(\pi_{\text{old}} \| \pi_\theta) \leq \delta$ after each update. Effective but requires expensive second-order optimisation (conjugate gradient).
3. **PPO** (Schulman et al., 2017): clip the probability ratio $r_t = \pi_\theta / \pi_{\text{old}}$ to $[1-\epsilon,\ 1+\epsilon]$ *inside* the objective. First-order only, simple to implement.

Our paper uses PPO — the simplest and most widely adopted approach. The next frames explain how the clipping mechanism works.

Step 2d: Why PPO Clipping Enables Multiple Epochs

PPO's Key Insight

By clipping the probability ratio at $\epsilon = 0.2$, the policy can only change by $\pm 20\%$ per update.

This has three important consequences:

1. **Conservative enough** to prevent catastrophic collapses — even with a noisy advantage estimate, one bad gradient step can only shift probabilities by 20%, not from 10% to 90%.
2. **Permissive enough** to allow steady learning — over many updates, the policy can still change dramatically (20% per step compounds quickly).
3. **Safe for multiple epochs** — since each pass over the batch can only move the policy $\pm 20\%$, reusing the same batch 4 times won't cause divergence. This is critical for our fix: it's what makes `epochs=4` safe despite the small batch size.

Without clipping, multiple epochs would overfit to the batch — the policy would chase noisy advantages until it collapsed.

Step 3a: The Probability Ratio

PPO reformulates the policy gradient using the **probability ratio**:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

Interpretation:

- $r_t = 1$: the new policy assigns the *same* probability to action a_t from state s_t as the old policy. No change.
- $r_t = 1.5$: the new policy is 50% *more* likely to take this action.
- $r_t = 0.7$: the new policy is 30% *less* likely to take this action.

Why use a ratio? The vanilla gradient uses $\log \pi_\theta(a \mid s)$, which can produce arbitrarily large updates. The ratio r_t lets us directly measure "how much did the policy change?" and **limit it**.

Importance sampling: Since we collected data under π_{old} but want to improve π_θ, the ratio corrects for this mismatch:

Step 3b: The Clipped Surrogate Objective

The unconstrained objective $L^{IS} = \mathbb{E}[r_t A_t]$ can still produce large updates (if r_t becomes very large). PPO **clips** the ratio:

$$L^{CLIP}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,A_t,\ \mathrm{clip}(r_t(\theta),\,1{-}\epsilon,\,1{+}\epsilon)\,A_t\big)\Big], \qquad \epsilon = 0.2$$

Reading this equation:

- $r_t(\theta)A_t$: the **unclipped** objective — how much the policy improves
- $\mathrm{clip}(r_t, 0.8, 1.2) \cdot A_t$: the **clipped** version — caps the ratio so the policy can change by at most $\pm 20\%$
- $\min(\cdot, \cdot)$: take whichever gives a **smaller improvement** (the "pessimistic bound")

Why min**?** We want to be *conservative*: if the unclipped objective says "increase this action's probability by 50%" but the clipped version says "only allow 20%", we use the clipped (smaller) version.

Step 3c: Clipping — Case-by-Case Analysis

Case	What happens
$A_t > 0,\ r_t < 1.2$	Unclipped: gradient encourages increasing $\pi(a_t \mid s_t)$ normally
$A_t > 0,\ r_t \geq 1.2$	**Clipped**: r_t capped at 1.2. No further gradient to increase probability beyond 20%
$A_t < 0,\ r_t > 0.8$	Unclipped: gradient encourages decreasing $\pi(a_t \mid s_t)$ normally
$A_t < 0,\ r_t \leq 0.8$	**Clipped**: r_t floored at 0.8. No further gradient to decrease beyond 20%

Intuition:

- **Good action** ($A_t > 0$): we want to increase its probability, but not too fast $\Rightarrow$ cap at $+20\%$.
- **Bad action** ($A_t < 0$): we want to decrease its probability, but not too fast $\Rightarrow$ floor at -20%.
- In both cases, once the clip activates, **the gradient is zero** — no further update is

PPO Clipping: In Our Code

The actual implementation in `training.py` (lines 386–415):

```
logratio = newlogprob - b_logprobs[mb_inds]
ratio = logratio.exp()  # pi(a|s) / pi_old(a|s)

# Advantage normalisation (scale-invariant!)
mb_advantages = (mb_advantages - mb_advantages.mean()) / (
    mb_advantages.std() + 1e-8)

# Clipped policy loss
pg_loss1 = -mb_advantages * ratio
pg_loss2 = -mb_advantages * torch.clamp(
    ratio, 1 - clip_coef, 1 + clip_coef)     # clip_coef = 0.2
pg_loss = torch.max(pg_loss1, pg_loss2).mean()
```

PPO Clipping: A Numerical Example

One transition from our AC problem:

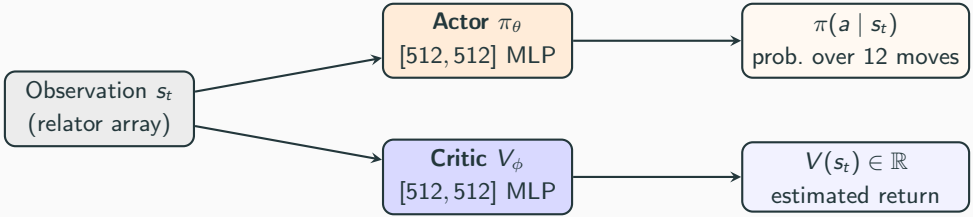
Quantity	Value	Meaning
$\pi_{\text{old}}(a_3 \mid s)$	0.10	Old policy: move 3 had 10% probability
$\pi_{\text{new}}(a_3 \mid s)$	0.15	After gradient step: now 15%
$r_t = 0.15/0.10$	1.50	50% increase — too much!
$\text{clip}(r_t, 0.8, 1.2)$	**1.20**	Capped at 20% increase
$A_t > 0$ (good action)		$\min(1.5A_t,\ 1.2A_t) = 1.2A_t$ $\Rightarrow$ gradient only "sees" 20% change

How `torch.max` implements the pessimistic bound:

- The loss is *negated*: `pg_loss1 = -A * r`, `pg_loss2 = -A * clip(r)`
- We *minimise* the loss, so `torch.max(loss1, loss2)` picks the *least negative* (= smallest improvement)

Step 4: Actor-Critic Architecture

PPO trains **two separate networks** simultaneously:



- **Actor loss** L^{CLIP}: push policy toward actions with positive advantage
- **Critic loss** $L^V = \mathrm{MSE}\big(V_\phi(s_t),\ G_t\big)$: predict actual returns from each state
- **Entropy bonus** $H[\pi]$: prevents premature convergence to a single action

Combined loss (optimized jointly with a single Adam optimizer):

$$\mathcal{L} = L^{CLIP} - \underbrace{c_{\text{ent}}}\cdot H[\pi] + \underbrace{c_{\text{vf}}}\cdot L^V$$

Step 5: Rollout Phase — Collecting a Batch

Before each PPO update, the agent collects data by running N environments in parallel for T steps each.

Definition: A **batch** is the set of all transitions collected in one rollout:

$$\text{batch\_size} = \underbrace{N}_{\text{envs}} \times \underbrace{T}_{\text{steps per env}}$$

Env 1	$(s_0, a_0, r_0, s_1),\ (s_1, a_1, r_1, s_2),\ \ldots,\ (s_{199}, a_{199}, r_{199}, s_{200})$
Env 2	$(s_0, a_0, r_0, s_1),\ (s_1, a_1, r_1, s_2),\ \ldots$
$\vdots$	$\vdots$
Env N	$(s_0, a_0, r_0, s_1),\ \ldots$

Each row is an independent AC presentation explored simultaneously. Each transition records: state, action (which of 12 moves), reward, and next state.

Our numbers: $N = 8,\ T = 200 \Rightarrow$ batch of **1,600 transitions** per update. The paper uses $N = 28 \Rightarrow$ **5,600 transitions**.

Step 6: Computing Advantages with GAE

After collecting a batch, we compute **how good each action was**.

Advantage $\hat{A}_t$: the difference between the actual return and what the critic predicted:

$$\hat{A}_t = G_t - V(s_t) \quad \text{where} \quad G_t = \sum_{k=0}^{T-t} \gamma^k \, r_{t+k}$$

Generalized Advantage Estimation (GAE) smooths this using parameter λ:

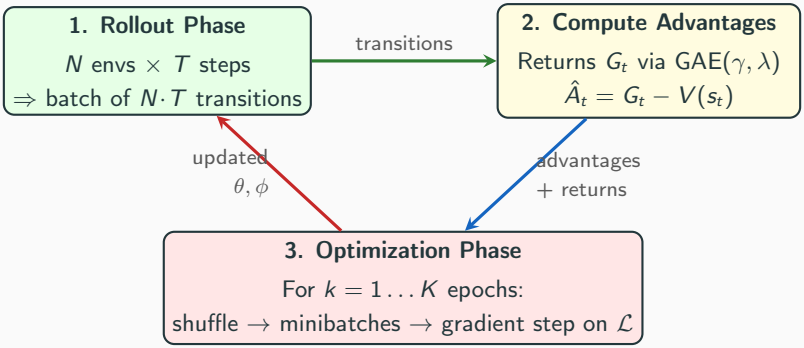
$$\hat{A}_t^{\text{GAE}} = \sum_{k=0}^{T-t} (\gamma\lambda)^k \, \delta_{t+k}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

- $\lambda = 0$: one-step TD error only (low variance, high bias)
- $\lambda = 1$: full Monte Carlo return (high variance, low bias)
- **Our setting:** $\lambda = 0.95$ — a standard compromise

Intuition: δ_t asks "was this step better or worse than the critic expected?" GAE averages these

Step 7: The Full PPO Training Loop

Putting it all together — training alternates between **collecting data** and **updating**:



PPO Training Loop: Concrete Numbers

Our setup vs. the paper:

Quantity	Paper	Ours	Meaning
N (parallel envs)	28	8	Simultaneous presentations
T (rollout length)	200	200	Steps per env before updating
Batch size	5,600	1,600	Transitions per PPO update
Minibatch size	1,400	400	Transitions per gradient step
K (epochs)	1	1 or **4**	Passes over batch per update
Total updates	17,857	6,250	In a 10M-step run

One PPO update cycle:

1. **Rollout:** 8 envs $\times$ 200 steps = 1,600 transitions ($\sim$1.2s)
2. **GAE:** compute advantages and returns ($\sim$0.01s)
3. **Optimize:** K epochs $\times$ 4 minibatches $\times$ 1 gradient step each ($\sim$0.1s for K=1, $\sim$0.4s for K=4)

The Role of Reward

The reward function defines **what the agent is trying to achieve**:

$$R(s_t, a_t, s_{t+1}) = \begin{cases} -\min(10,\ \text{length}(s_{t+1})) & \text{if length}(s_{t+1}) > 2 \\ +1000 & \text{if solved (length} = 2) \end{cases}$$

Why this design:

- **Per-step penalty** $-\min(10, \text{len})$: discourages making presentations longer. Capped at -10 to limit gradient variance.
- **Success bonus** $+1000$: massive reward for reaching the trivial state.

What the critic learns from rewards:

The critic estimates $V(s) = \mathbb{E}[\text{future return from state } s]$.

- States *near* a solution: high $V(s)$ (large expected future reward).
- States *far* from any solution: low $V(s)$ (only penalties ahead).

The Role of Batch

What "batch" means: Before each PPO update, the agent collects a **batch** of experience: $\text{batch\_size} = N_{\text{envs}} \times T_{\text{steps}}$.

An **environment (env)** is one instance of the AC problem being explored. Each env runs independently — a different starting presentation, taking different actions, generating its own trajectory.

Why batch size matters: The policy gradient is estimated by averaging over the batch. Larger batch ⇒ more accurate gradient estimate.

$$\text{Gradient noise} \propto \frac{1}{\sqrt{\text{batch\_size}}}$$

	Envs (N)	**Steps (T)**	**Batch**	**Relative noise**
Paper	28	200	5,600	1.0×
Ours	8	200	1,600	1.87×

Why the Paper Uses $N = 28$ Environments

The paper (Appendix A) specifies $N = 28$ parallel actors. This is **not arbitrary**: with $T = 200$ and 10^8 total timesteps,

$$\text{updates} = \frac{10^8}{28 \times 200} = 17{,}857 \quad \text{vs.} \quad \frac{10^8}{8 \times 200} = 62{,}500 \text{ (ours)}$$

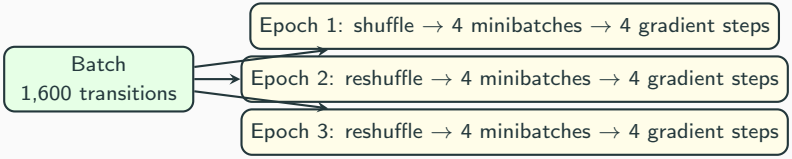
The paper makes 3.5× fewer but 3.5× **higher-quality** updates.

Why more envs helps beyond just batch size:

- More **diverse presentations** per batch — with 28 envs, each batch sees 28 different starting states (vs. our 8).
- Better **curriculum coverage**: the training curriculum samples 25% solved, 75% unsolved. With 28 envs, ∼7 envs see solved presentations and ∼21 see unsolved — a richer gradient signal.
- **Parallelism**: the paper likely ran on multi-core CPU or GPU, so 28 envs added minimal wall-clock time.

The Role of Epochs

What "epoch" means in PPO: After collecting one batch, how many times do we *reuse* it for gradient updates?



- **epochs = 1**: see each transition once, compute gradient, discard batch.
- **epochs = 4**: four passes with different minibatch groupings $\Rightarrow$ extract $4\times$ more signal from the same data.
- **Safety valve:** `target_kl = 0.01` stops early if the policy diverges too far from the rollout policy.

Key insight: epochs compensate for small batches.

Scaled-Down Subset

Subset Design: 147 Instances

Method: Stratified sampling with `every_k=10` from each $(n, |w|)$ cell.

$\lvert w\rvert$	**Full cell**	**Sampled**
1–3	2 each	1 each
4	6	1
5	18	2
6	42	5
7	98	10
Per n-row:		**21**
Total (7×21):		**147**

Why this works:

- Every $(n, |w|)$ cell has ≥ 1 instance
  $\Rightarrow$ full difficulty coverage
- Easy baseline ($|w| \leq 3$, 100% greedy) retained for sanity checks
- Hard frontier ($|w| = 6{,}7$ at high n) has 5–10 instances for statistical power
- Reuses existing code: `load_stratified_subset(d, every_k=10)`

12.4% of full dataset. Expected greedy baseline: $\sim$46% (matches full dataset rate).

What Do n and $|w|$ Control?

The Miller–Schupp series $\mathrm{MS}(n, w)$ is parameterised by two values:

Parameter n: controls the first relator.

$$r_0 = x^{-1} y^n x y^{-(n+1)}$$

Higher $n \Rightarrow$ longer $r_0 \Rightarrow$ larger search space. From the paper (Section 3.3, Figure 1): solve rate **decreases monotonically with** n.

Parameter $|w|$: controls the second relator.

$$r_1 = x^{-1} w \quad \text{where } w \text{ has zero exponent sum on } x$$

Higher $|w| \Rightarrow$ more complex $w \Rightarrow$ more "entangled" presentation.

Key Theoretical Result (Theorem 2, p13)

$\mathrm{MS}(1, w)$ is **AC-trivial for all** w. This explains why greedy solves 100% of n=1 instances — these are provably easy. The paper also notes that greedy solved *all* n=1 instances in the full dataset.

The Role of $|w|$: Second Relator Complexity

While n controls the first relator, $|w|$ controls the **combinatorial complexity** of the second relator $r_1 = x^{-1}\, w$.

What does $|w|$ encode?

- w is a word in $\{x, x^{-1}, y, y^{-1}\}$ with **zero exponent sum on** x (i.e., $\#x = \#x^{-1}$ in w).
- $|w| = 1$: only $w = y$ or $w = y^{-1}$ (2 choices per n).
- $|w| = 7$: 98 distinct words after removing cyclic permutations.
- Higher $|w| \Rightarrow$ more ways to "entangle" the generators $\Rightarrow$ more complex interaction between r_0 and r_1.

The Role of $|w|$: Examples and Paper Observations

Concrete examples (all with $n = 2$, so $r_0 = x^{-1}y^2xy^{-3}$):

$\lvert w\rvert$	w	$r_1 = x^{-1}\,w$	**Greedy?**
1	y	$x^{-1}\,y$	Solved
3	$yx^{-1}x$	$x^{-1}\,y\,x^{-1}\,x$	Solved
5	$y^2x^{-1}yy^{-1}x$	$x^{-1}\,y^2\,x^{-1}\,y\,y^{-1}\,x$	Depends
7	(98 possibilities)		Many unsolved

Paper observation (Section 3.3, Figure 1b):

- Greedy solve rate is *less sensitive* to $|w|$ than to n.
- At fixed n, larger $|w|$ sometimes *helps* — longer words offer more free-reduction opportunities when concatenated.
- But the **hardest unsolved instances** tend to have both high n *and* high $|w|$ — the combination creates deeply entangled presentations that require long sequences of AC

Full Dataset Distribution (Reference)

Each $(n, |w|)$ cell has identical counts across all n values:

$n \backslash \lvert w \rvert$	1	2	3	4	5	6	7	**Total**
1	2	2	2	6	18	42	98	170
2	2	2	2	6	18	42	98	170
3	2	2	2	6	18	42	98	170
4	2	2	2	6	18	42	98	170
5	2	2	2	6	18	42	98	170
6	2	2	2	6	18	42	98	170
7	2	2	2	6	18	42	98	170
Total	14	14	14	42	126	294	686	**1190**

- n controls first relator complexity ($r_0 = x^{-1} y^n x y^{-(n+1)}$)
- $|w|$ controls second relator length ($r_1 = x^{-1} w$)

Classical Baselines on Subset

Classical Baselines: Subset vs. Full Dataset

Algorithm	Subset	Subset %	Full %	Match?
Greedy (10^6 nodes)	**71** / 147	**48.3%**	46.6%	✓
BFS (10^6 nodes)	42 / 147	28.6%	23.4%	≈

Subset is representative: solve rates match full dataset within ∼2–5 percentage points.

BFS slightly higher on subset (28.6% vs. 23.4%) because stratified sampling overrepresents small cells ($|w| \leq 3$) where both algorithms solve 100%.

Commands:

```
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 --workers 4 --every-k 10
python -m ac_solver.search.miller_schupp.evaluate \
```

Greedy Baseline: Per-$(n, |w|)$ Solve Rates on Subset

$n \backslash |w|$	1	2	3	4	5	6	7	**Row**
1	1/1	1/1	1/1	1/1	2/2	5/5	10/10	**21/21**
2	1/1	1/1	1/1	1/1	2/2	4/5	6/10	16/21
3	1/1	1/1	1/1	0/1	1/2	4/5	3/10	11/21
4	1/1	1/1	1/1	0/1	0/2	1/5	2/10	6/21
5	1/1	1/1	1/1	0/1	0/2	2/5	4/10	9/21
6	1/1	1/1	1/1	0/1	0/2	1/5	0/10	4/21
7	1/1	1/1	1/1	0/1	0/2	1/5	0/10	4/21

- $|w| \leq 3$: 100% solved (all cells have size 1, all trivially easy)
- $n = 1$: 100% solved regardless of $|w|$ (MS(1,w) is AC-trivial for all w)
- **Hard frontier** ($n \geq 4$, $|w| \geq 4$): steep drop-off, 0–40% solve rate
- PPO must beat this baseline to demonstrate value

BFS Baseline: Per-$(n, |w|)$ Solve Rates on Subset

$n \backslash \lvert w \rvert$	1	2	3	4	5	6	7	**Row**
1	1/1	1/1	1/1	1/1	2/2	2/5	5/10	13/21
2	1/1	1/1	1/1	0/1	0/2	3/5	4/10	10/21
3	1/1	1/1	1/1	0/1	0/2	1/5	0/10	4/21
4	1/1	1/1	1/1	0/1	0/2	1/5	0/10	4/21
5	1/1	1/1	1/1	0/1	0/2	1/5	0/10	4/21
6	1/1	1/1	1/1	0/1	0/2	1/5	0/10	4/21
7	1/1	1/1	1/1	0/1	0/2	0/5	0/10	3/21

- BFS drops off faster than greedy: 42/147 vs. 71/147
- BFS guarantees **shortest paths** but exhausts 10^6-node budget sooner
- For $n \geq 3$: BFS solves only $|w| \leq 3$ cells (with rare exceptions)
- **Gap:** greedy solves 29 instances that BFS cannot within budget

PPO Ablation Design

Ablation Design: 3 Runs, 2 Factors

Sequential ablation design: isolate each factor one at a time.

Run	Label	Reward fix	Epochs	Purpose
0	`diag_baseline`	No	1	Control (broken config)
1	`diag_reward_fix`	Yes	1	Isolate reward fix
2	`diag_reward_epochs`	Yes	4	Reward fix + data reuse

- **Factor 1: Reward scaling** (`-skip-reward-division`). Removes `rewards /= 14400` so PPO sees $[-10, +1000]$ matching the paper.
- **Factor 2:** `update_epochs` ($1 \rightarrow 4$). Paper uses 1 with batch = 5,600; we have batch = 1,600. With epochs = 4, effective data = 6,400 $>$ paper's 5,600.
- **Budget:** $3 \times$ 10M steps = 30M total.
- **Single seed** (s=1): looking for large effects, not small statistical differences.

Fixed Hyperparameters (Same Across All Runs)

Parameter	Value	Source	Rationale
`num_envs`	8	Hardware	Mac limit; held constant
`num_steps`	200	Paper §4.4	Constant-horizon setup
`batch_size`	1,600	8×200	Constrained by above
`nodes`	$[512, 512]$	Paper App. A	2-layer MLP
γ	0.999	Paper App. A	Effective horizon $\sim$1000
λ_{GAE}	0.95	Paper App. A	Bias-variance trade-off
ϵ_{clip}	0.2	Paper App. A	PPO clipping
c_{ent}	0.01	Paper App. A	Exploration bonus
c_{vf}	0.5	Paper App. A	Value loss coefficient
`target_kl`	0.01	Paper App. A	Early stop in epoch loop
`max_grad_norm`	0.5	Paper App. A	Gradient clipping
Adam ϵ	10^{-5}	Paper App. A	Numerical stability
`repeat_solved_prob`	0.25	Paper §4.4	Curriculum
`seed`	1	Convention	Fixed for reproducibility
LR schedule	linear$\rightarrow$0	Paper App. A	Same across all 3 runs

Varied Factors: Justification

Factor 1: Reward scaling (`-skip-reward-division`)

- Removes `rewards /= 14400` from `training.py`
- PPO sees $[-10, +1000]$ matching the paper exactly
- Expected to help the critic learn meaningful value differences

Factor 2: `update_epochs` ($1 \rightarrow 4$)

- Paper uses 1; standard PPO uses 3–10
- At batch size 1,600 (small), single-pass extracts minimal signal
- With epochs $= 4$: effective data $= 1{,}600 \times 4 = 6{,}400$, exceeding the paper's 5,600 per update
- `target_kl=0.01` prevents overfitting per epoch

Why Not LR Schedule?

We did not run cosine LR experiments (originally planned as Runs 3 and 4). At 10M steps

Success Criteria (Before Looking at Results)

Criterion	Threshold	Interpretation
Reward fix works	Run 1 $>$ 2$\times$ Run 0	Division was the cause
Epochs help	Run 2 $>$ Run 1 by $>$30%	Data reuse matters
Combined viable	Run 2 $>$ 5/147 (3.4%)	Worth extending to 50M
Approaching paper	Run 2 $>$ 15/147 (10%)	Could match paper at scale
Baselines hold	Greedy $\approx$ 46%	Subset is representative

✓ **Baselines confirmed:** Greedy 48.3%, BFS 28.6%

Why Epochs = 4 is the Critical Fix

Key Insight: `norm_adv` Makes Reward Scale Less Important

Recall that the PPO policy gradient uses *normalised* advantages (`norm_adv=True`):

$$\hat{A}_t^{\text{norm}} = \frac{\hat{A}_t - \bar{A}}{\sigma_A + 10^{-8}} \qquad \Longrightarrow \qquad \text{mean} \approx 0,\ \text{std} \approx 1 \ \textbf{regardless of reward scale}$$

	Before norm		After norm	
	$\bar{A}$	σ_A	$\bar{A}^{\text{norm}}$	σ_A^{norm}
Run 0 ($\div$14400)	-0.001	0.11	≈ 0	≈ 1
Run 1 (no $\div$)	-174	37	≈ 0	≈ 1

The actor receives *identical* gradient magnitudes in both runs.

Implication

The reward fix helps the *critic* learn real value differences, but the *policy* is insensitive to reward scale because normalisation removes it. The dominant bottleneck was **data reuse** (`epochs`), not reward scale. This explains why Run 1 (reward fix only) $\approx$ Run 0 in evaluation.

Why `epochs=4` is the Critical Fix

Mathematical analysis: The signal-to-noise ratio (SNR) of the policy gradient scales with the effective number of samples:

$$\text{SNR}_{\text{policy}} \propto \frac{\sqrt{N_{\text{eff}}}}{\sigma_A} \quad \text{where} \quad N_{\text{eff}} \approx \text{batch} \times \text{epochs}$$

Configuration	Batch	Epochs	N_{eff}	Relative SNR
Paper (28 envs)	5,600	1	5,600	1.00×
Run 0/1 (8 envs)	1,600	1	1,600	0.53×
Run 2 (8 envs, ep=4)	1,600	**4**	**6,400**	1.07×

Key point: With epochs = 1, our gradient SNR is only 53% of the paper's. The policy gradient is too noisy to learn — effectively a random walk in parameter space.

With epochs = 4, SNR *exceeds* the paper's setup. This is the cheapest possible fix: zero additional environment steps, only extra gradient computation.

How σ_A Is Computed: Advantage Normalisation in Detail

The advantage standard deviation σ_A in the SNR formula comes from `norm_adv` (line 404 of `training.py`):

$$\hat{A}_t^{\text{norm}} = \frac{\hat{A}_t - \bar{A}}{\sigma_A + 10^{-8}} \quad \text{where} \quad \bar{A} = \frac{1}{M}\sum_{i=1}^{M} \hat{A}_i, \;\; \sigma_A = \sqrt{\frac{1}{M}\sum_{i=1}^{M} (\hat{A}_i - \bar{A})^2}$$

Here $M = 400$ (minibatch size). The statistics are computed **per minibatch**, not over the full batch.

Advantage Normalisation: Worked Example and Implications

Worked example (one minibatch of 400 transitions at 10M):

	Run 0 ($\div 14400$)	**Run 1** (paper scale)	
Raw advantages $\hat{A}_i$	$[-0.15, +0.12]$	$[-250, +180]$	Range
$\bar{A}$	-0.001	-160	Mean
σ_A	0.063	36.2	Std dev
After norm: $\bar{A}^{\text{norm}}$	≈ 0	≈ 0	**Same**
After norm: σ_A^{norm}	≈ 1	≈ 1	**Same**

Normalisation removes scale differences. What it cannot fix:

- If all raw advantages are nearly identical (critic gives constant V), normalisation **amplifies noise into signal** $\Rightarrow$ random gradient directions.
- The σ_A denominator *scales up* tiny fluctuations into unit variance — a minibatch where $\hat{A}_i \in [-0.002, +0.001]$ becomes $\hat{A}_i^{\text{norm}} \in [-1.3, +0.7]$.
- More epochs help: averaging over 16 reshuffled minibatch gradients (instead of 4) cancels out

The Epoch Mechanism: Multiple Gradient Steps

Each epoch **reshuffles** the batch into new minibatches, creating diverse gradient estimates from the same data:

Epoch	Minibatch composition
1	Shuffle 1,600 transitions $\rightarrow$ 4 minibatches of 400 $\rightarrow$ 4 gradient steps
2	**Reshuffle** $\rightarrow$ different groupings of 400 $\rightarrow$ 4 new gradient steps
3	Reshuffle $\rightarrow$ yet different groupings $\rightarrow$ 4 more gradient steps
4	Reshuffle $\rightarrow$ final groupings $\rightarrow$ 4 final gradient steps
Total	**16 gradient steps** per PPO update (vs. 4 with epochs $= 1$)

Why reshuffling matters:

- Each minibatch of 400 sees a *different random subset* of the 1,600 transitions
- Different subsets produce different gradient directions
- Averaging over 16 steps (instead of 4) reduces gradient variance by $\sim 2\times$

Epochs vs. Batch Size: A Mathematical Equivalence

Claim: K epochs with batch $B \approx$ batch $B \cdot K$ in terms of gradient variance.

With i.i.d. samples, gradient variance scales as:

$$\mathrm{Var}[\nabla \mathcal{L}] \propto \frac{\sigma^2}{N}$$

	Batch	**Epochs**	**Grad steps**	**Variance**
Batch $= B \cdot K$, ep $= 1$	BK	1	BK/M	σ^2/BK
Batch $= B$, ep $= K$	B	K	$K \cdot B/M$	$\approx \sigma^2/BK$

Caveat — staleness:

- With a larger batch, all samples are fresh (collected under current policy $\pi_{\theta_{\text{old}}}$)
- With multiple epochs, later epochs use slightly *stale* data: the policy has already moved from $\pi_{\theta_{\text{old}}}$
- PPO's clipping mechanism limits this drift (next slide)

PPO Clipping Prevents Epoch Overfitting

How PPO stays safe with multiple epochs:

The probability ratio $r_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is **clipped** to $[1-\epsilon,\ 1+\epsilon]$ with $\epsilon = 0.2$:

$$L^{CLIP} = \min\big(r_t A_t,\ \ \text{clip}(r_t, 0.8, 1.2)\, A_t\big)$$

Once r_t hits the clip boundary, further gradient steps on that transition produce **zero gradient** — the clipping acts as a per-sample learning rate schedule.

Additional safety: `target_kl = 0.01`

After each epoch, we compute the approximate KL divergence:

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \approx \mathbb{E}_t\left[\frac{(r_t - 1)^2}{2}\right]$$

If $D_{\text{KL}} > 0.01$, the epoch loop **terminates early**, discarding remaining epochs for this update.

Result: At 700K steps, Run 2 shows clipfrac $= 0.0016$ and approx_kl $= 0.0013$ — the policy moves gently even with 4 epochs. Most transitions are not clipped; the policy is safely

Empirical Evidence: Epochs Change Everything

Training metrics comparison at ∼700K steps:

Metric	Run 0	Run 1	Run 2	Interpretation
entropy	2.484	2.483	**2.065**	Policy specialising
expl. var	−1.42	−0.009	−0.000	Critic learning
top-1 prob	9.0%	9.2%	**31.0%**	Confident actions
clipfrac	0.000	0.000	0.0016	Active clipping
approx_kl	0.000	0.000	0.0013	Policy updating
policy_loss	+0.00005	+0.00004	−0.0148	Meaningful gradient
noop rate	60.3%	59.4%	71.1%	Learned to avoid bad moves

The contrast is dramatic:

- Runs 0 and 1 (epochs = 1): entropy ≈ 2.48 (near-uniform over 12 actions), top-1 ≈ 9%, zero clipping. No learning is happening.
- Run 2 (epochs = 4): entropy dropped to 2.06, top-1 jumped to 31%, clipping is active.

PPO Training Results: All Three Runs

Training Metrics: Run 0 vs. Run 1 vs. Run 2 at 700K

Metric	Run 0	Run 1	Run 2	Key difference
entropy	2.484	2.483	2.065	Run 2: -0.42 (policy specialising)
expl. var	-1.42	-0.009	-0.000	Run 1 fixed critic; Run 2 same
top-1 prob	9.0%	9.2%	31.0%	Run 2: 3.4$\times$ more confident
clipfrac	0.000	0.000	0.0016	Only Run 2 has active clipping
approx_kl	0.000	0.000	0.0013	Only Run 2 updates policy
policy_loss	$+0.00005$	$+0.00004$	-0.0148	Run 2: 300$\times$ larger magnitude
noop rate	60.3%	59.4%	71.1%	Run 2 avoids length-increasing moves

Interpretation:

- **Run 0 vs. Run 1:** The reward fix improved the critic (expl. var: $-1.42 \rightarrow -0.009$) but the *policy* is unchanged (entropy, top-1, clipfrac all identical).
- **Run 1 vs. Run 2:** Adding epochs $= 4$ *transforms* the policy. The same reward signal, processed 4$\times$ more thoroughly, produces genuine learning.
- **Conclusion:** The reward fix was necessary (critic needs real signal) but epochs were the *sufficient*

Training Metrics at 10M Steps

Metric	Run 0	Run 1	Run 2	Key difference
entropy	2.449	2.467	**1.308**	Run 2: sharp specialisation
expl. var	−1.75	−0.000	+0.005	Run 2 critic slightly positive
top-1 prob	12.9%	11.7%	**52.8%**	Run 2: majority action >50%
clipfrac	0.000	0.000	0.000	Run 2 clipping done (converged)
noop rate	61.3%	61.8%	50.6%	Run 2 *reduced* noops

Run 2 at 10M vs. 700K:

- Entropy: 2.065 → 1.308 — policy became much more decisive
- Top-1: 31% → 52.8% — dominant action selected majority of the time
- Noop rate: 71.1% → 50.6% — initially conservative, now taking more risks
- Clipfrac: 0.0016 → 0.000 — policy updates have stabilised

Runs 0 and 1 show no meaningful change from 700K to 10M — still near-uniform policies with zero clipping.

Why the Reward Fix Didn't Help the *Policy*

The PPO loss function has three terms:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{policy}}}_{\text{actor gradient}} - c_{\text{ent}} \cdot H[\pi] + c_{\text{vf}} \cdot \underbrace{\mathcal{L}_{\text{value}}}_{\text{critic gradient}}$$

The **policy gradient** uses *normalised* advantages (`norm_adv=True`):

$$\hat{A}_t^{\text{norm}} = \frac{\hat{A}_t - \bar{A}}{\sigma_A + 10^{-8}} \quad \Longrightarrow \quad \text{mean} \approx 0,\ \text{std} \approx 1 \ \textbf{regardless of reward scale}$$

The actor receives *identical* gradient magnitudes in both Runs 0 and 1.

Advantage normalisation makes the policy update **scale-invariant** in the reward.

The reward fix *cannot* improve the actor through this mechanism.

Where Does Reward Scale Actually Matter?

Reward scale → critic signal quality → advantage *direction* (not scale) → policy gradient usefulness. `norm_adv` removes scale; it cannot remove noise.

A noisy advantage, once normalised, is still noisy.

Where Does Reward Scale Actually Matter?

Actor and critic have **separate parameters** (paper footnote 11 confirms this). Adam maintains per-parameter adaptive LR $\Rightarrow$ actor and critic gradients **do not interfere**, even with a single optimiser.

Reward scale matters for the **critic's learning quality**, not the actor's gradient magnitude:

	$\div 14{,}400$ **(our code)**	**Paper scale**
Returns	$G_t \in [-0.14, +0.07]$	$G_t \in [-2000, +1000]$
Critic target spread	~ 0.2 (all states look the same)	~ 3000 (huge differences)
Critic learning	"Predict ≈ 0.04" fits everything $\Rightarrow$ expl. var $+0.66$ but no per-state signal	Must learn genuine $V(s)$ differences
Advantage quality	Noise (critic gives constant baseline)	Meaningful signal

The Real Mechanism

Reward scale $\rightarrow$ critic signal quality $\rightarrow$ advantage *direction* (not scale) $\rightarrow$ policy gradient

PPO Evaluation Results

Evaluation Results: The Full Picture

Algorithm	Mode	Solved	Rate
Greedy (10^6 nodes)	—	**71**/147	**48.3%**
BFS (10^6 nodes)	—	42/147	28.6%
Run 0 (baseline) 1M	det / sto×5	0 / 1	0.0% / 0.7%
Run 0 (baseline) 5M	det / sto×5	0 / 1	0.0% / 0.7%
Run 0 (baseline) 10M	det / sto×5	0 / 2	0.0% / 1.4%
Run 1 (reward fix) 1M	det / sto×5	0 / 1	0.0% / 0.7%
Run 1 (reward fix) 5M	det / sto×5	0 / 1	0.0% / 0.7%
Run 1 (reward fix) 10M	det / sto×5	0 / 1	0.0% / 0.7%
Run 2 (rwd+ep4) 1M	det / sto×5	0 / 2	0.0% / 1.4%
Run 2 (rwd+ep4) 5M	det / sto×5	0 / 4	0.0% / 2.7%
Run 2 (rwd+ep4) 10M	det / sto×5	**8 / 10**	**5.4% / 6.8%**

Run 2 at 10M: the breakthrough

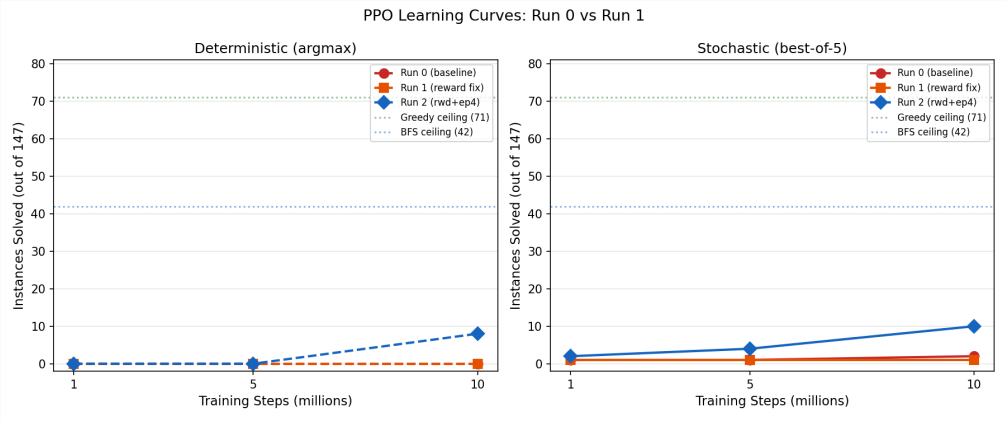
The Breakthrough: Run 2 in Detail

	1M		5M		10M	
Run	**det**	**sto5**	**det**	**sto5**	**det**	**sto5**
Run 0 (baseline)	0	1	0	1	0	2
Run 1 (rwd fix)	0	1	0	1	0	1
Run 2 (rwd+ep4)	0	2	0	4	**8**	**10**

Key observations:

- **Runs 0 and 1:** 0 deterministic solves at any checkpoint. Stochastic best-of-5 solves 1–2 $\approx$ random walk baseline.
- **Run 2 at 10M:** 8 deterministic + 2 additional stochastic = 10 total. This is a real learned policy, not random chance.
- **Scaling with training:** Run 2 improves monotonically ($0 \to 0 \to 8$ det; $2 \to 4 \to 10$ sto5). More training = more solves.

Learning Curves: PPO vs. Classical Baselines

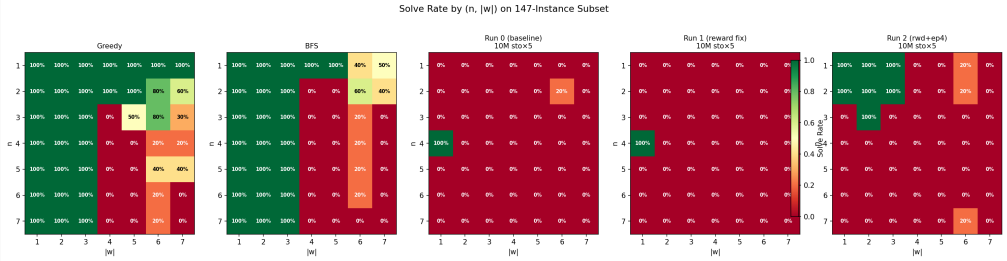


PPO solve counts at 1M, 5M, 10M checkpoints (deterministic and stochastic best-of-5).

Run 2 shows clear upward trajectory; Runs 0 and 1 remain near zero.

(Note: figure may need regeneration to include Run 2 data.)

Heatmaps: Classical vs. PPO Solve Rates

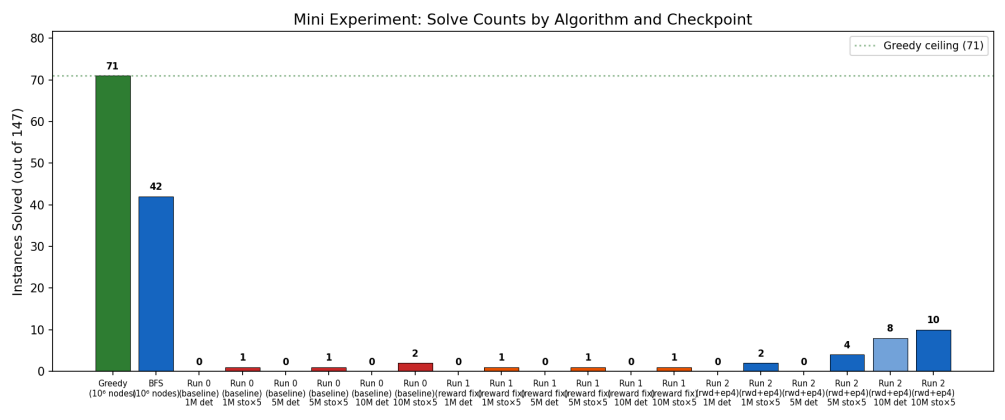


Per-$(n, |w|)$ solve rates. Greedy and BFS show clear structure.

Run 2 begins to show structure at low n and $|w|$.

(Note: figure may need regeneration to include Run 2 data.)

Solve Counts: Bar Chart Overview



All algorithms and checkpoints on the 147-instance subset.

Run 2 at 10M is the first PPO configuration to show visible bars.

Training Solve Rate vs. Evaluation: A Warning

	Training "solved"	Eval (sto×5)
Run 0 at 10M	1190 (cumulative)	2/147 (1.4%)
Run 1 at 10M	1190 (cumulative)	1/147 (0.7%)
Run 2 at 10M	1190 (cumulative)	10/147 (6.8%)
Previous 100M run	1190/1190	32/1190 (2.7%)

Why the gap?

- Training uses **stochastic** sampling — a 200-step random walk often stumbles onto easy solutions
- "Total solved" is **cumulative** — counts *every* instance solved across all episodes
- Evaluation uses **deterministic** (argmax) or controlled stochastic rollouts
- Training solve rate is not a reliable performance metric

Analysis and Next Steps

Success Criteria: Results

Criterion	Threshold	Actual	Status
Reward fix works	Run 1 $>$ 2$\times$ Run 0	1 vs. 2 (sto5)	Not met
Epochs help	Run 2 $>$ Run 1 by 30%	10 vs. 1 (10$\times$!)	✓✓Met
Combined viable	Run 2 $>$ 5/147	10/147	✓Met
Approaching paper	Run 2 $>$ 15/147 (10%)	10/147 (6.8%)	Partial
Baselines hold	Greedy $\approx$ 46%	48.3%	✓Met

Summary of findings:

1. The reward fix alone is **insufficient** — it helps the critic but `norm_adv` makes the policy insensitive to reward scale.
2. **Epochs $= 4$ is the critical fix.** It compensates for our 3.5$\times$ smaller batch and enables the policy to actually learn.
3. The combination (reward fix + epochs) is **viable** at 10M steps: 8 deterministic, 10 stochastic solves — a genuine signal worth extending.

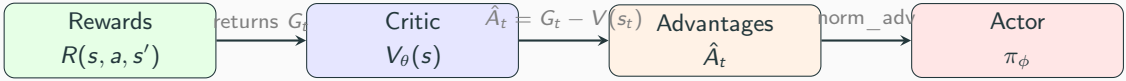
Paper vs. Our Setup: The Complete Gap Analysis

The paper (Appendix A) uses CleanRL with `norm_adv=True`, separate actor/critic, single Adam optimiser — **identical to our implementation**.

Parameter	**Paper**	**Ours (Run 2)**	**Gap**
`num_envs`	28	8	3.5× fewer
`batch_size`	5,600	1,600	3.5× smaller
`update_epochs`	1	4	Compensates batch gap
Effective data/update	5,600	6,400	1.14× more
`total_timesteps`	10^8	10^7	10× fewer
`rewards /= 14400`	**No**	No	Fixed
`norm_adv`	True	True	Same
Result	431/1190 (36%)	10/147 (6.8%)	Training budget

Remaining gap: primarily **training budget** (10×) and **dataset size** (147 vs. 1190). The hyperparameter gaps have been closed.

The Learning Pipeline: Where Each Fix Acts



1. **Reward scale** (Fix 1) → critic can learn meaningful $V(s)$ differences → advantages point in the right *direction*
2. **Epochs** (Fix 2) → extract enough signal from each batch to overcome gradient noise → policy actually updates
3. **Both needed:** Fix 1 without Fix 2 = good advantages, but too noisy to use (Run 1). Fix 2 without Fix 1 = not tested, but likely weak (critic still can't distinguish states).

`norm_adv` removes advantage *scale*; it cannot remove *noise*.

Why Epochs Worked: The Complete Picture

The causal chain:

1. **Small batch** (1,600 vs. 5,600): each minibatch of 400 transitions produces a noisy gradient estimate.
2. **epochs** $= 1$: only 4 gradient steps per PPO update. The noise dominates — the policy takes a near-random walk in parameter space. Clipfrac $= 0$, approx_kl $= 0$: the policy is not moving at all.
3. **epochs** $= 4$: 16 gradient steps per PPO update. The noise averages out — the policy moves in a consistent direction. Clipfrac $= 0.0016$: clipping activates, proving the policy is updating.
4. **Result**: entropy drops (policy specialises), top-1 increases (confident actions), and eventually the policy solves 8 instances deterministically.

The Insight

With a small batch, the gradient SNR per step is too low for 1 epoch to extract meaningful

Next Steps: Extend Run 2 to 50M Steps

Why 50M:

- Run 2 is still improving at 10M (monotonic across all checkpoints)
- The paper trains for 10^8 steps — we have used only 10^7
- At 10M, Run 2 achieves 6.8% vs. the paper's 36% at 10^8
- The scaling trajectory ($2 \rightarrow 4 \rightarrow 10$ sto5) suggests substantial headroom

Recommended configuration:

Parameter	Value
Base config	Run 2 (reward fix + epochs = 4)
Total steps	50M
LR schedule	Linear$\rightarrow$0 (or cosine with floor — test both)
Checkpoints	Every 5M for evaluation curve

Decision rule:

Appendix: CLI Commands i

Classical baselines:

```
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn greedy --max-nodes 1000000 --workers 4 --every-k 10 \
    --output results/greedy_subset_k10.csv
python -m ac_solver.search.miller_schupp.evaluate \
    --search-fn bfs --max-nodes 1000000 --workers 2 --every-k 10 \
    --output results/bfs_subset_k10.csv
```

PPO training:

Appendix: CLI Commands ii

```
# Run 0: baseline
python -m ac_solver.agents.ppo --preset mac --seed 1 --exp-name diag_baseline
# Run 1: reward fix
python -m ac_solver.agents.ppo --preset mac --seed 1 \
    --skip-reward-division --exp-name diag_reward_fix
# Run 2: reward fix + epochs=4
python -m ac_solver.agents.ppo --preset mac --seed 1 \
    --skip-reward-division --update-epochs 4 --exp-name diag_reward_epochs
```

Appendix: Experiment File Locations

Artifact	Path
Experiment spec	`spec/2026-04-03-scaled-experiment.md`
Greedy subset CSV	`results/greedy_subset_k10.csv`
BFS subset CSV	`results/bfs_subset_k10.csv`
Run 0 checkpoints	`out/diag_baseline_ppo-.../ckpt_*.pt`
Run 0 metrics	`results/mini_experiment/run0_baseline_metrics.csv`
Run 1 checkpoints	`out/diag_reward_fix_ppo-.../ckpt_*.pt`
Run 1 metrics	`results/mini_experiment/run1_reward_fix_metrics.csv`
Run 2 checkpoints	`out/diag_reward_epochs_ppo-.../ckpt_*.pt`
Run 2 metrics	`results/mini_experiment/run2_reward_epochs_metrics.csv`
This presentation	`presentations/mini_reproducing_results_slides.tex`